

BUG FIX · 2.12 LINE

Filestore restores a stale `fb1k` for `MaxMsgsPer>1` with FIFO removal

Why per-subject message lookups could fail after a restart on nats-server 2.12 — and why 2.14 is structurally immune.

PR: [nats-io/nats-server#8285](https://github.com/nats-io/nats-server/pull/8285) File: `server/filestore.go`

Affects: file-backed JetStream, `MaxMsgsPer > 1` Trigger: FIFO eviction + restart

TL;DR

The filestore keeps a per-subject pointer, `fb1k` (“first block”), so it can find a subject’s oldest message without scanning every block on disk. On the **2.12** line, when a subject with several copies is trimmed down to a single message by **FIFO eviction**, `fb1k` can be left pointing at a block that no longer holds any message for that subject. Because of how `index.db` serializes single-message subjects, this **stale pointer gets persisted**, and after a **restart** lookups start scanning from the wrong place and **fail to find the message**. PR #8285 restores a one-line correction (`info.fb1k = info.lb1k`) that fixes the pointer before it is persisted. On **2.14** the on-disk format always stores both pointers, so the bug cannot occur.

Background: what `fb1k` and `lb1k` are for

A file-backed stream is split into many message **blocks** on disk (`1.blk`, `2.blk`, ...). To answer “what is the first/oldest sequence for subject X?” quickly, the store keeps a small per-subject info record (`psi`) in memory:

```
type psi struct {
    total uint64 // how many messages this subject currently has
    fb1k  uint32 // first block that holds a message for this subject
```

```
lblk uint32 // last block that holds a message for this subject
}
```

Lookups use this pair as a search window instead of scanning the whole store

(`server/filestore.go`, `firstSeqForSubj`):

```
// See if we can optimize where we start.
start, stop := fs.blks[0].index, fs.lmb.index
if info, ok := fs.psim.Find(stringToBytes(subj)); ok {
    start, stop = info.fblk, info.lblk // <-- only scan [fblk .. lblk]
}
```

So correctness hinges on one invariant: **fblk** must point at (or before) the block that actually contains the subject's oldest surviving message. If **fblk** points past it, the message is invisible to the scan.

The key detail: how single-message subjects are persisted

When stream state is written to `index.db`, the 2.12 format does **not** store **lblk** for a subject that has only one message — it stores **total** and **fblk**, and on restore it reconstructs **lblk** = **fblk**:

```
// decode (2.12 behavior)
psi := psi{total: readU64(), fblk: uint32(readU64())}
if psi.total > 1 {
    psi.lblk = uint32(readU64())
} else {
    psi.lblk = psi.fblk // single-msg subject: lblk is *derived* from fblk
}
```

This is the heart of it: for a one-message subject, **fblk** is the only pointer that survives a restart, and **lblk** is rebuilt from it. So if **fblk** is wrong when **total == 1**, then after a restart *both* pointers are wrong, and the search window **[fblk .. lblk]** collapses onto the wrong block.

Why the pointer goes stale: lazy fblk updates

On removal, the store deliberately does **not** recompute fblk right away — it's a lazy optimization, normally corrected later during a lookup (removePerSubject):

```
// We do not update sense of fblk here but will do so when we resolve during lookup.
if info, ok := fs.psim.Find(bsubj); ok {
    info.total--
    // ... fblk left unchanged ...
}
```

That laziness is fine as long as a lookup happens before the state is persisted. The dangerous combination is MaxMsgsPer > 1 (a subject legitimately living across several blocks) trimmed by **FIFO eviction** (oldest-first), so the surviving message ends up in a later block than fblk — and then the state is written out and the server restarts before any lookup corrects it.

Walkthrough

- 1 Subject foo with MaxMsgsPer = 3 has copies in blocks 1, 4, and 7.

psi = {total: 3, fblk: 1, lblk: 7}



red outline = fblk · green outline = lblk

- 2 FIFO eviction removes the two oldest copies (blocks 1 and 4). Only the copy in block 7 survives.

total drops 3 → 1, but fblk is **not** recomputed (lazy).



psi = {total: 1, fblk: 1 (stale!), lblk: 7}

- 3 Stream state is flushed to index.db. Because total == 1, only fblk = 1 is written; lblk is not persisted.

- 4 Server restarts. On restore: fblk = 1 and lblk = fblk = 1.

psi = {total: 1, fblk: 1, lblk: 1} — search window is now just block 1.

5 Lookup for `foo` scans only block 1, finds nothing, and the message in block 7 is effectively lost.

The fix (PR #8285)

The PR restores a block that had been removed in 2.12.7. When FIFO removal brings a subject down to a single message, it eagerly corrects `fblk` to `lblk` — the block where the survivor actually is — so the value that gets persisted is correct:

```
// If we only ever store one/last message for a subject,  
// can correct the first block to where we've just written.  
if info != nil && info.total == 1 && mmp == 1 {  
    info.fblk = info.lblk  
}
```

Now at step 2 above, `fblk` becomes 7, the persisted pointer is correct, and after restart the lookup scans block 7 and finds the message.

Test coverage

The PR adds a regression test in `server/filestore_test.go` that builds a subject spanning multiple blocks under `MaxMsgsPer>1`, drives FIFO removal down to one message, restarts the store, and asserts the message is still retrievable (i.e. `fblk` is not stale).

Why 2.14 is no longer affected

The 2.14 line fixes this at the **format level** rather than by patching the removal path. The newer `index.db` encoder **always** writes both `fblk` and `lblk` for every subject, regardless of `total`:

```
// encode (2.14) - always persists both pointers  
buf = binary.AppendUvarint(buf, psi.total)  
buf = binary.AppendUvarint(buf, uint64(psi.fblk))  
buf = binary.AppendUvarint(buf, uint64(psi.lblk)) // always written
```

```
// decode (2.14) – reads lblk whenever the format version supports it
psi := psi{total: readU64(), fblk: uint32(readU64())}
if psi.total > 1 || version >= 4 {
    psi.lblk = uint32(readU64())
} else {
    psi.lblk = psi.fblk
}
```

Two things follow from this:

- `lblk` is no longer *derived* from `fblk` on restore — it's read directly from disk and remains correct (e.g. block 7) even if `fblk` is stale.
- Because `lblk` survives accurately, the lazy-`fblk` design works as intended: after restart the search window `[fblk .. lblk]` still includes the surviving message's block, the lookup finds it, and the same lookup then self-corrects `fblk` back to the right block.

In short: on 2.12 the survivor's location had to be encoded into `fblk` (because `lblk` wasn't persisted for single-message subjects), so a stale `fblk` was fatal. On 2.14 the location is persisted independently in `lblk`, so a temporarily-stale `fblk` is harmless and self-healing. PR #8285 is the backport-shaped workaround that gives 2.12 the same guarantee without changing its on-disk format.

Version comparison

Behavior	2.12 (pre-fix, ≥2.12.7)	2.12 + PR #8285	2.14+
Persists <code>lblk</code> for single-msg subject	No (derived from <code>fblk</code>)	No (derived from <code>fblk</code>)	Yes (always)
Corrects <code>fblk</code> before persisting on FIFO trim-to-one	No	Yes	N/A (not needed)
Message retrievable after restart (<code>MaxMsgsPer>1</code> , FIFO)	No — bug	Yes	Yes

Scope & limitations

- **2.12 only.** The fix targets the 2.12 release line; 2.14+ already handles this via the format change.
- **FIFO removal path only.** The correction is applied on the limits/eviction path. Other removal patterns on the 2.12 line (e.g. an explicit `RemoveMsg` on the newest message rather than oldest-first eviction) can still leave a stale pointer — a known, documented constraint of 2.12.
- **Low risk.** It restores previously-shipped behavior plus a regression test; no on-disk format change.

Generated explainer for [nats-io/nats-server#8285](https://github.com/nats-io/nats-server/pull/8285). Code references: `server/filestore.go`

(`firstSeqForSubj`, `removePerSubject`, `index.db encode/decode`). For authoritative behavior, consult the PR and the source.